

UPWORK PROPOSAL

ASSISTANT

Comprehensive Database System Design Document

Technical Specification Manual — Version 2.0



Project Development Group Members:

Sr. No	Group Member Name	Roll Number / ID
1	Tayyab Shehzad	NUM-BSCS-2024-77
2	Najeeba	NUM-BSCS-2024-61
3	Kashif Ali	NUM-BSCS-2024-29

Department of Computer Sciences
Namal University
Mianwali, Pakistan

Document Submission Date: 14th June, 2026

Contents

1	Project Overview & Requirement Analysis	4
1.1	1.1 Introduction	4
1.2	1.2 System Scope and Domain Rules	4
1.3	1.3 Functional Requirements	5
1.4	1.4 Non-Functional Requirements	5
2	Conceptual Database Design	6
2.1	2.1 Entity Relationship Architecture Summary	6
2.2	2.2 Data Dictionary Specifications	6
2.2.1	2.2.1 1. Person Table	6
2.2.2	2.2.2 2. Freelancer Table	7
2.2.3	2.2.3 3. Client Table	8
2.2.4	2.2.4 4. Admin Table	8
2.2.5	2.2.5 5. Skills Table	9
2.2.6	2.2.6 6. Freelancer Skill Map Table	9
2.2.7	2.2.7 7. Job Posting Table	11
2.2.8	2.2.8 8. Job Skill Map Table	11
2.2.9	2.2.9 9. Proposal Template Table	11
2.2.10	2.2.10 10. Reliability Score Table	12
2.2.11	2.2.11 11. Freelancer Analytics Table	12
2.2.12	2.2.12 12. Generated Proposals Table	12
3	Logical Database Design	19
3.1	3.1 Relational Schema Mapping Rules	19
3.2	3.2 Functional Dependencies	19

4	Implementation	21
4.1	4.1 Database Engine and Environment Settings	21
4.2	4.2 Database Programmability Objects	21
4.3	4.3 Interface Payload Mapping	21
	Appendix: AI Usage and Prompts	24

List of Tables

2.1	Person Table Schema Definition	7
2.2	Freelancer Table Schema Definition	8
2.3	Client Table Schema Definition	9
2.4	Admin Table Schema Definition	10
2.5	Skills Table Schema Definition	10
2.6	Freelancer Skill Map Table Schema Definition	11
2.7	Job Posting Table Schema Definition	13
2.8	Job Skill Map Table Schema Definition	14
2.9	Proposal Template Table Schema Definition	15
2.10	Reliability Score Table Schema Definition	16
2.11	Freelancer Analytics Table Schema Definition	17
2.12	Generated Proposals Table Schema Definition	18
4.1	Database Programmability Objects Directory	22

Chapter 1

Project Overview & Requirement Analysis

1.1 1.1 Introduction

The modern freelance ecosystem demands unprecedented velocity and positioning precision from independent contractors bidding on open client contracts. Platforms like Upwork present highly competitive, fast-moving target environments where manually composing customized cover letters delays proposal delivery pipelines, dropping hiring conversion metrics.

The Upwork Proposal Assistant is engineered to eliminate these marketplace friction boundaries. By implementing a highly structured relational database infrastructure, the system tracks historically effective proposal text modules, matches real-time client postings with professional skills directories, computes client safety thresholds, and archives generated artifacts to establish a high-throughput, automated bidding support tool.

1.2 1.2 System Scope and Domain Rules

The backend framework enforces strict architectural guidelines to handle multi-tenant isolation, automated analytical computation, and transactional audit trails:

- **Identity Specialization Layers:** High-level user properties are consolidated into a base entity. Concrete downstream operations branch disjointly into specific paths for contractors (tracking billing histories), employers (handling evaluation histories), and internal monitoring agents.
- **Marketplace Skill Mapping Bridges:** Individual technical proficiencies must map fluidly to both freelancer accounts and incoming job descriptions. This complex

connectivity is decomposed into dedicated associative mapping vectors to prevent array field storage exceptions.

- **Background Automated Risk Evaluation:** The platform monitors client marketplace performance. Triggers automatically process success rates and update security risk tiers, warning users about bad-faith actors or low-conversion biddings before financial resources are spent.

1.3 1.3 Functional Requirements

1. **FR-1 (User Identity Management):** Enforce strict string constraints and unique email tracking maps across all onboarding registrations.
2. **FR-2 (Job Posting Ingestion):** Allow employers to submit detailed project briefs bounded by experience indicators and explicit financial budget caps.
3. **FR-3 (Template Repository Control):** Empower administrative profiles to maintain reusable boilerplate records while systematically accumulating dynamic usage analytics metrics.
4. **FR-4 (Analytical Automation):** Automatically recalculate operational performance metrics and client risk indices whenever underlying transaction rows transition.

1.4 1.4 Non-Functional Requirements

1. **NFR-1 (Referential Integrity Enforcement):** Enforce consistent cascading constraint behaviors across all related multi-table drop paths to stop orphan row accumulation.
2. **NFR-2 (Physical Storage Normalization):** Structure all database schemas to satisfy Third Normal Form (3NF) requirements, eliminating delete, insert, and update operational anomalies.
3. **NFR-3 (Transactional Optimization):** Implement storage routines directly within the database engine to isolate processing logic and guarantee low network interaction latency.

Chapter 2

Conceptual Database Design

2.1 2.1 Entity Relationship Architecture Summary

The conceptual architecture structures operational data vectors cleanly to ensure continuous transaction processing performance. It isolates general demographic elements into a centralized index, allowing specific downstream profiles to inherit baseline properties securely. The tracking layout connects client briefs, skills indexes, administrative boilerplate configurations, and automated background analysis rows. This approach ensures data remains clean and atomic during high-frequency platform modifications.

2.2 2.2 Data Dictionary Specifications

The specific domain types, key configurations, and structural validation attributes defining the relational database schema are cataloged within the tables below. Every single column features an expanded, deeply detailed technical description defining its validation parameters, mathematical constraint scopes, and structural purpose:

2.2.1 2.2.1 1. Person Table

The **Person** table functions as the base foundational supertype entity within the platform's security and profile layer. It strips away overlapping personal identity attributes from localized transactional roles, ensuring the database avoids data footprint bloat. Keeping identity values isolated in a single relation guarantees 1NF atomicity and prevents updating errors when global items change.

Table 2.1: Person Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Expanded Description / Constraints
<code>person_id</code>	INT	PK	NOT NULL	System-generated auto-increment sequence primary key. Serves as the immutable unique pointer across the generalized schema entity tree.
<code>f_name</code>	VARCHAR(50) —		NOT NULL	User’s validated legal first name. Restrained via frontend application triggers to block numeric elements, emojis, or punctuation symbols.
<code>Lname</code>	VARCHAR(50) —		NOT NULL	User’s validated legal last name. Enforces an absolute non-empty text row entry constraint to support profile rendering.
<code>email</code>	VARCHAR(100)—		UNIQUE	Primary electronic mail tracking handle. Enforces strict uniqueness index bounds and checks for regex pattern syntax compliance (<code>char@domain.ext</code>).
<code>country</code>	VARCHAR(50) —		NULL	Optional geographical registration string. Maps directly to a validated internal country code directory to support location filtering.
<code>upwork_profile_url</code>	VARCHAR(255)—		UNIQUE	Verified external portfolio address link string. Enforced via uniqueness indexes to block duplicate target account cloning across user records.

2.2.2 2.2.2 2. Freelancer Table

The **Freelancer** table represents a strict specialized subtype partition branched directly from the core **Person** index. Its primary key (`freelancer_id`) is explicitly mapped as a foreign key referencing **Person**(`person_id`) under a **ON DELETE CASCADE** policy. This 1:1 specialization layout removes non-prime operational variables—such as specific hourly pricing boundaries and industry tenure lengths—away from general system profiles, meeting 3NF requirements since all fields depend exclusively on the candidate key.

Table 2.2: Freelancer Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Expanded Description / Constraints
<code>freelancer_id</code>	INT	PK, FK	NOT NULL	Shared subtype unique identifier key. Connects directly to Person (<code>person_id</code>) with cascade deletes to maintain referencing integrity.
<code>password</code>	VARCHAR(255)	—	NOT NULL	Security access string array. Holds cryptographically protected password strings processed using industry-standard Argon2id hashing configurations.
<code>hourly_rate</code>	DECIMAL(6,2)	—	NOT NULL	Service fee rate assigned for contractor billing. Bound by a strict validation rule checking that numerical records remain above \$0.00.
<code>experience_years</code>	INT	—	NULL	Total industry tenure length tracker. Checked by backend parameters to maintain logical inputs strictly between 0 and 60 years.

2.2.3 2.2.3 3. Client Table

The **Client** relation records operational attributes specific to enterprise or independent employers posting project briefs. To prevent database design errors, client metrics (such as aggregate hiring ratios and qualitative textual feedback historical logs) are strictly kept out of global tracking sheets. The table’s primary key serves as a direct referential anchor to the master **Person** table, ensuring clean relational boundaries where business metrics do not interfere with developer profiles.

2.2.4 2.2.4 4. Admin Table

The **Admin** framework manages security clearances, operational states, and governance privileges across platform elements. Keeping administrative rows in an isolated subtype structure means internal system parameters (like moderator assignments and duty states) remain securely separated from public-facing user configurations. This approach simplifies access controls and prevents structural or security oversights.

Table 2.3: Client Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Expanded Description / Constraints
<code>client_id</code>	INT	PK, FK	NOT NULL	Subtype anchor key mapping to <code>Person(person_id)</code> . Enforces complete cascading deletion rules across downstream transaction layers.
<code>company_name</code>	VARCHAR(100)	—	NULL	Legal corporate business title string. Left as an optional field to support both solo employers and large corporate entities.
<code>success_rate</code>	DECIMAL(5,2)	—	NULL	Rolling metric monitoring historical job closure rates. Maintained via automatic procedure updates bounded tightly between 0.00 and 100.00.
<code>feedback</code>	TEXT	—	NULL	Long-form text block field reserved for qualitative reviews, contractor feedback summaries, and overall platform history tracking.

2.2.5 2.2.5 5. Skills Table

The `Skills` table acts as a master lookup directory that catalogs technical competencies on the platform. It enforces a strict unique index constraint on the `skill_name` attribute string to prevent spelling variations or duplicate entries from corrupting the lookup index. This layout guarantees a highly normalized repository for clean data tracking.

2.2.6 2.2.6 6. Freelancer Skill Map Table

This associative entity defines structural mapping configurations between users and technical competencies, cleanly resolving the complex many-to-many relationship linking freelancers to multiple skills. This intersection mapping pattern avoids storage anomalies and array violations by recording dependencies strictly as individual binary key-pair rows, fulfilling 2NF requirements.

Table 2.4: Admin Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Expanded Description / Constraints
admin_id	INT	PK, FK	NOT NULL	Governance tracking anchor referencing Person(person_id) . Ensures platform support parameters remain separated from typical consumer spaces.
role	VARCHAR(50)	—	NOT NULL	Explicit operations access assignment string. Restricted via application validation rules to: 'SuperAdmin', 'Moderator', 'Analyst'.
joined_date	DATE	—	NOT NULL	System date record capturing the initialization of admin status. Enforces storage compliance using the formal ISO YYYY-MM-DD standard format.
status	VARCHAR(20)	—	NOT NULL	Administrative state toggle parameter. Restricted by engine validation checks to: 'Active', 'Suspended', or 'OnLeave'.

Table 2.5: Skills Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Expanded Description / Constraints
skill_id	INT	PK	NOT NULL	Unique key sequence token. Acts as the static master directory link handling technical skill definitions across mapping matrices.
skill_name	VARCHAR(100)	—	UNIQUE	Distinct index lookup text string (e.g., 'MySQL', 'Python'). Managed by case-insensitive matching rules to block duplicate rows.
skill_level	VARCHAR(30)	—	NOT NULL	Capability tier classification. Checked at the database engine level to restrict options strictly to: 'Beginner', 'Intermediate', or 'Expert'.

Table 2.6: Freelancer Skill Map Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Expanded Description / Constraints
<code>freelancer_id</code>	INT	PK, FK	NOT NULL	Part of composite primary key sequence. Links to Freelancer (<code>freelancer_id</code>) using deep cascade rules to handle user profile deletions cleanly.
<code>skill_id</code>	INT	PK, FK	NOT NULL	Part of composite primary key sequence. Links to Skills (<code>skill_id</code>) with restrictive boundaries to block dropping skills that are currently in active use.

2.2.7 2.2.7 7. Job Posting Table

The **Job_Posting** table catalogs structural details for individual contract listings uploaded by verified employers. The relation adheres completely to 3NF standards because every single descriptive non-prime variable—including budget limits, funding structures, and experience criteria checks—depends entirely on the primary surrogate identifier `job_id`. This prevents transitive dependencies across employer entities.

2.2.8 2.2.8 8. Job Skill Map Table

The **Job_Skill_Map** associative relation links specific technical competencies to active job postings. Breaking down many-to-many connections into simple key-pair assignments ensures that when required skills are modified, the operations remain localized, fast, and completely free of unintended data mutations.

2.2.9 2.2.9 9. Proposal Template Table

The **Proposal_Template** repository tracks standard template configurations crafted by administrative moderators. Isolating layout components from the main application log sheets optimizes storage and ensures 3NF compliance. User profiles can reuse these boilerplates simply by referencing the unique `template_id` key, keeping overall data overhead low.

2.2.10 2.2.10 10. Reliability Score Table

The `Reliability_Score` entity logs client metric snapshots for real-time risk profiling. Keeping automated calculations outside of primary profiling tables ensures high 3NF performance. This isolation guarantees that background analytical score adjustments do not inadvertently lock or corrupt primary user records.

2.2.11 2.2.11 11. Freelancer Analytics Table

The `Freelancer_Analytics` table maintains execution summaries of user conversions. Storing these dynamic performance stats apart from core profiles helps avoid structural anomalies. This pattern ensures that rapid analytical tracking updates can run smoothly without risking data locks on authentication rows.

2.2.12 2.2.12 12. Generated Proposals Table

The `Generated_Proposals` table records transactional data from cover letter assembly loops. It coordinates foreign keys from four distinct parent entities into a unified structure. Because all final cover letter content blocks and status flags depend entirely on the unique primary key `proposal_id`, the table completely avoids transitive anomalies and satisfies all 3NF boundaries.

Table 2.7: Job Posting Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Expanded Description / Constraints
job_id	INT	PK	NOT NULL	Primary surrogate tracking index key tracking contract records. Automatically steps up sequentially on every incoming post request.
client_id	INT	FK	NOT NULL	References <code>Client(client_id)</code> . Implements a restricted delete rule to block closing manager profiles while they run open contracts.
job_title	VARCHAR(150)—		NOT NULL	Headline title summarizing the contract listing. Cleaned by input boundaries to eliminate raw code fragments or injection vectors.
job_description	TEXT	—	NOT NULL	Long-form text field capturing project milestones, required methodologies, and explicit candidate technical deliverables.
experience_level	VARCHAR(20) —		NOT NULL	Targeted talent tier classification constraint. Regulated strictly via domain validation checks to: 'Entry', 'Mid', or 'Expert'.
job_type	VARCHAR(20) —		NOT NULL	Pricing financial structural toggle. Constrained by verification parameters to accept exclusively 'Fixed-price' or 'Hourly' strategies.
project_duration	VARCHAR(50) —		NULL	Optional text string summarizing estimated assignment lengths (e.g., '1 to 3 months') to support frontend filtering.
budget_amount	DECIMAL(10,2)—		NOT NULL	Maximum allocated funding pool capacity. Bound by a strict check constraint enforcing that currency values remain strictly above \$0.00.
posted_date	TIMESTAMP	—	NOT NULL	System generated logging stamp. Automatically records the exact server time transaction when the job row transitions to an active state.
proposal_count	INT	—	DEFAULT 0	Live application counter tracking incoming bids. Increments automatically via insertion listener triggers on the proposal tables.

Table 2.8: Job Skill Map Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Expanded Description / Constraints
<code>job_id</code>	INT	PK, FK	NOT NULL	Part of composite primary key. References <code>Job_Posting(job_id)</code> and cascades updates automatically to wipe orphan mapping rows cleanly.
<code>skill_id</code>	INT	PK, FK	NOT NULL	Part of composite primary key. References <code>Skills(skill_id)</code> and uses strict foreign key limits to keep required platform taxonomies accurate.

Table 2.9: Proposal Template Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Expanded Description / Constraints
template_id	INT	PK	NOT NULL	Primary key identifier tracking shared layout elements. Indexes standardized boilerplate records across user selection views.
admin_id	INT	FK	NULL	References Admin(admin_id). Uses a set-null policy to preserve boilerplate structures even if the creating administrative account is removed.
template_name	VARCHAR(100)—		NOT NULL	Descriptive name indexing the specific layout profile (e.g., 'Cloud Infrastructure Design'). Used for rapid dashboard filtering.
category	VARCHAR(50) —		NOT NULL	Operational group label text string used to bundle similar items together (e.g., 'Web Frontend', 'Data Analytics').
template_content	TEXT	—	NOT NULL	Master boilerplate body layout text block. Features parsing anchors processed by text generation layers during cover letter assembly loops.
usage_count	INT	—	DEFAULT 0	Tally monitoring structural usage. Updated continuously via database engine insert procedures to tracking platform asset value.
success_rate	DECIMAL(5,2) —		NULL	Conversion performance tracking metric. Calculates the long-term effectiveness of boilerplate items in winning successful hires.

Table 2.10: Reliability Score Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Expanded Description / Constraints
<code>score_id</code>	INT	PK	NOT NULL	Primary system tracking index. Isolates analytical score calculations away from live client profiling records to maintain fast query processing.
<code>client_id</code>	INT	FK	NOT NULL	References <code>Client(client_id)</code> . Employs cascading deletions to automatically purge metric snapshots if an employer profile is dropped.
<code>score</code>	DECIMAL(5,2)	—	NOT NULL	Calculated reliability metric value. Bound by constraints strictly between 0.00 and 100.00 to guarantee calculation range safety.
<code>risk_level</code>	VARCHAR(20)	—	NOT NULL	Categorized evaluation band. Enforced via database check boundaries to capture precisely: 'Low', 'Medium', or 'High'.
<code>recommendation_level</code>	VARCHAR(50)	—	NOT NULL	Operational workflow advisory text string. Automatically outputs values like 'Highly Recommended' or 'Avoid' based on scoring rules.
<code>calculated_date</code>	TIMESTAMP	—	NOT NULL	Detailed execution log timestamp capturing exactly when the procedural computation engine updated the client's risk values.

Table 2.11: Freelancer Analytics Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Expanded Description / Constraints
<code>analytics_id</code>	INT	PK	NOT NULL	Unique index key assigned to decouple real-time rolling calculations from primary user registration and login tables.
<code>freelancer_id</code>	INT	FK	NOT NULL	References <code>Freelancer(freelancer_id)</code> . Regulated by unique indexes to restrict data mapping precisely to a single tracking row per user.
<code>total_proposals</code>	INT	—	DEFAULT 0	Tally of all submitted job applications. Incremented continuously through database transaction insert listeners.
<code>accepted_proposals</code>	INT	—	DEFAULT 0	Tally monitoring positive client hiring responses. Maintained through back-end update triggers when contract proposals transition to active status.
<code>rejected_proposals</code>	INT	—	DEFAULT 0	Tally monitoring passed or unapproved applications across account bid histories. Updated via status change triggers.
<code>success_rate</code>	DECIMAL(5,2)	—	DEFAULT 0.00	Computed percentage ratio tracking profile conversion performance. Enforced via checks to maintain values between 0.00 and 100.00.

Table 2.12: Generated Proposals Table Schema Definition

Attribute Name	Data Type	Key	Null/Unique	Expanded Description / Constraints
proposal_id	INT	PK	NOT NULL	Primary system transaction key tracking cover letter generations. Steps up auto-incrementally on every dynamic composition run.
freelancer_id	INT	FK	NOT NULL	References Freelancer(freelancer_id) . Employs a restrictive delete constraint rule to preserve historical bidding logs for platform data audits.
client_id	INT	FK	NOT NULL	References Client(client_id) . Connects individual cover letters directly to original job publishers to streamline messaging pipelines.
job_id	INT	FK	NOT NULL	References Job_Posting(job_id) . Links the submission to specific contract records, implementing cascading rules if listing components are removed.
template_id	INT	FK	NOT NULL	References Proposal_Template(template_id) . Identifies the specific boilerplate asset style used during dynamic letter text assembly.
proposal_content	TEXT	—	NOT NULL	The final customized cover letter body text, compiled by parsing engine scripts to match the target project scope.
generated_date	TIMESTAMP	—	NOT NULL	Automated execution record marker capturing the exact server time transaction when the generation thread finished processing text strings.
proposal_status	VARCHAR(30)	—	NOT NULL	Status tracking parameter constraint. Engine constraints strictly limit text entries to: 'Draft', 'Submitted', 'Accepted', or 'Rejected'.
submission_date	TIMESTAMP	—	NULL	Optional server timestamp documenting the exact moment files were pushed to external APIs. Remains null during drafting phases.

Chapter 3

Logical Database Design

3.1 3.1 Relational Schema Mapping Rules

The operational architecture translates the conceptual model into clean, physical tables based on clear relational design parameters. The 1:1 specialization line connecting the **Person** table to downstream components uses a shared primary key constraint strategy, eliminating blank row vectors while avoiding highly sparse tables. Double-sided many-to-many linkages are decomposed using dedicated intersection mapping elements featuring composite primary arrangements. This pattern protects data granularity while naturally satisfying 3NF normalization rules.

3.2 3.2 Functional Dependencies

All data fields map directly and explicitly to primary composite candidate keys to satisfy Third Normal Form (3NF) requirements:

- $\text{person_id} \rightarrow \{\text{f_name}, \text{Lname}, \text{email}, \text{country}, \text{upwork_profile_url}\}$
- $\text{freelancer_id} \rightarrow \{\text{password}, \text{hourly_rate}, \text{experience_years}\}$
- $\text{client_id} \rightarrow \{\text{company_name}, \text{success_rate}, \text{feedback}\}$
- $\text{admin_id} \rightarrow \{\text{role}, \text{joined_date}, \text{status}\}$
- $\text{skill_id} \rightarrow \{\text{skill_name}, \text{skill_level}\}$
- $\text{job_id} \rightarrow \{\text{client_id}, \text{job_title}, \text{job_description}, \text{experience_level}, \text{job_type}, \text{project_duration}, \text{budget_amount}\}$
- $\text{template_id} \rightarrow \{\text{admin_id}, \text{template_name}, \text{category}, \text{template_content}, \text{usage_count}, \text{success_rate}\}$
- $\text{score_id} \rightarrow \{\text{client_id}, \text{score}, \text{risk_level}, \text{recommendation_level}, \text{calculated_date}\}$

- analytics_id $\rightarrow$ {freelancer_id, total_proposals, accepted_proposals, rejected_proposals, success_rate}
- proposal_id $\rightarrow$ {freelancer_id, client_id, job_id, template_id, proposal_content, generated_date, proposal_status}

Because every single non-key attribute relies fully on the primary candidate key, the structural schema remains completely free of partial or transitive dependencies, satisfying 3NF boundaries.

Chapter 4

Implementation

4.1 Database Engine and Environment Settings

The physical relational architecture runs on a local deployment of the MySQL 8.0 Relational Engine using the InnoDB transactional storage driver. InnoDB provides complete ACID (Atomicity, Consistency, Isolation, Durability) validation compliance, executing row-level write-locking mechanics during active simultaneous write streams. The database engine collation uses `utf8mb4_unicode_ci` configurations to ensure URL links and profile names are saved securely without text truncation bugs.

4.2 Database Programmability Objects

To automate performance calculations and minimize connection roundtrip overhead, database routines and trigger rules handle core evaluation workflows directly within the storage engine:

4.3 Interface Payload Mapping

The application layer communicates securely via decoupled database connection drivers. Relational data arrays seamlessly pass payloads between database instances and front-end interface views without structural formatting bottlenecks or formatting errors. Standardizing shared primary references allows application connectors to query metrics instantly, preparing the pre-built tables to support stable, continuous system querying.

Table 4.1: Database Programmability Objects Directory

Sr.	Object Name	Object Type	Description	Input / Output Parameters
01	Calculate_Reliability	Stored Procedure	Evaluates client success rates against policy data, updates risks, and appends a score row.	Input: <code>c_id</code> INT, <code>raw_score</code> DECIMAL Output: Inserts to <code>Reliability_Score</code> .
02	Increment_Template_Usage	Trigger	Automatically increments the template usage counter when a new proposal row is added.	Event: AFTER INSERT on <code>Generated_Proposals</code> table.
03	Sync_Freelancer_Metrics	Stored Procedure	Automatically recalculates totals, acceptance counters, and success rate values for a freelancer.	Input: <code>f_id</code> INT Output: Modifies row inside <code>Freelancer_Analytics</code> .

Bibliography

- [1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.
- [2] MySQL Reference Manual, “Stored Procedures, Functions, and Triggers Infrastructure,” Oracle Corp, 2026.

Appendix: AI Usage and Prompts

In compliance with project design evaluation guidelines, the development prompts used with generative artificial intelligence models are documented below:

- **Prompt 1 (Schema Refactoring):** “Review my Upwork database design layout attributes. Fix any potential decimal data type truncation bottlenecks within the job posting financial budgets and client reliability performance score range values to ensure it safely handles up to 100% metric values without runtime crashes.”
- **Prompt 2 (Transactional Scaling):** “Generate a fully expanded, interconnected relational seed script mapping out 2 distinct administrative roles (one for system user metrics analysis, one for outgoing template tracking) along with more than 20 rows of realistic transactional mock records per operational table to test structural performance.”
- **Prompt 3 (Documentation Mapping):** “Map my updated relational set structures into a comprehensive logical validation report structured under formal Chapter 3 and Chapter 4 sections following Namal University project templates.”